

Race Condition in C++

What is a Race Condition?

A race condition occurs when two or more threads access shared data at the same time, and at least one of them modifies the data, leading to unpredictable or incorrect behavior.

Why is it Called a 'Race'?

Because threads are racing to access or modify the data, and the outcome depends on the timing of their execution. The result becomes non-deterministic and hard to debug.

Example Without Synchronization (Race Condition)

```
#include <iostream>

#include <thread>

int counter = 0;

void increment() {
    for (int i = 0; i < 10000; ++i)
        ++counter; // NOT thread-safe
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);

    t1.join();
    t2.join();

    std::cout << "Final counter: " << counter << std::endl;
    return 0;
}
```

Race Condition in C++

Expected vs Actual Output

Expected Output:

Final counter: 20000

Actual Output (may vary):

Final counter: 18429

Because both threads increment 'counter' without synchronization, they interfere with each other's operations.

Fixing Race Condition Using Mutex

```
#include <iostream>
```

```
#include <thread>
```

```
#include <mutex>
```

```
int counter = 0;
```

```
std::mutex mtx;
```

```
void increment() {
```

```
    for (int i = 0; i < 10000; ++i) {
```

```
        std::lock_guard<std::mutex> lock(mtx);
```

```
        ++counter;
```

```
    }
```

```
}
```

```
int main() {
```

```
    std::thread t1(increment);
```

```
    std::thread t2(increment);
```

```
    t1.join();
```

```
    t2.join();
```

Race Condition in C++

```
std::cout << "Final counter: " << counter << std::endl;  
return 0;  
}
```

Summary

- Race condition = simultaneous access/modification without sync
- Causes random and hard-to-debug bugs
- Fix using mutex, atomic operations, or other synchronization techniques